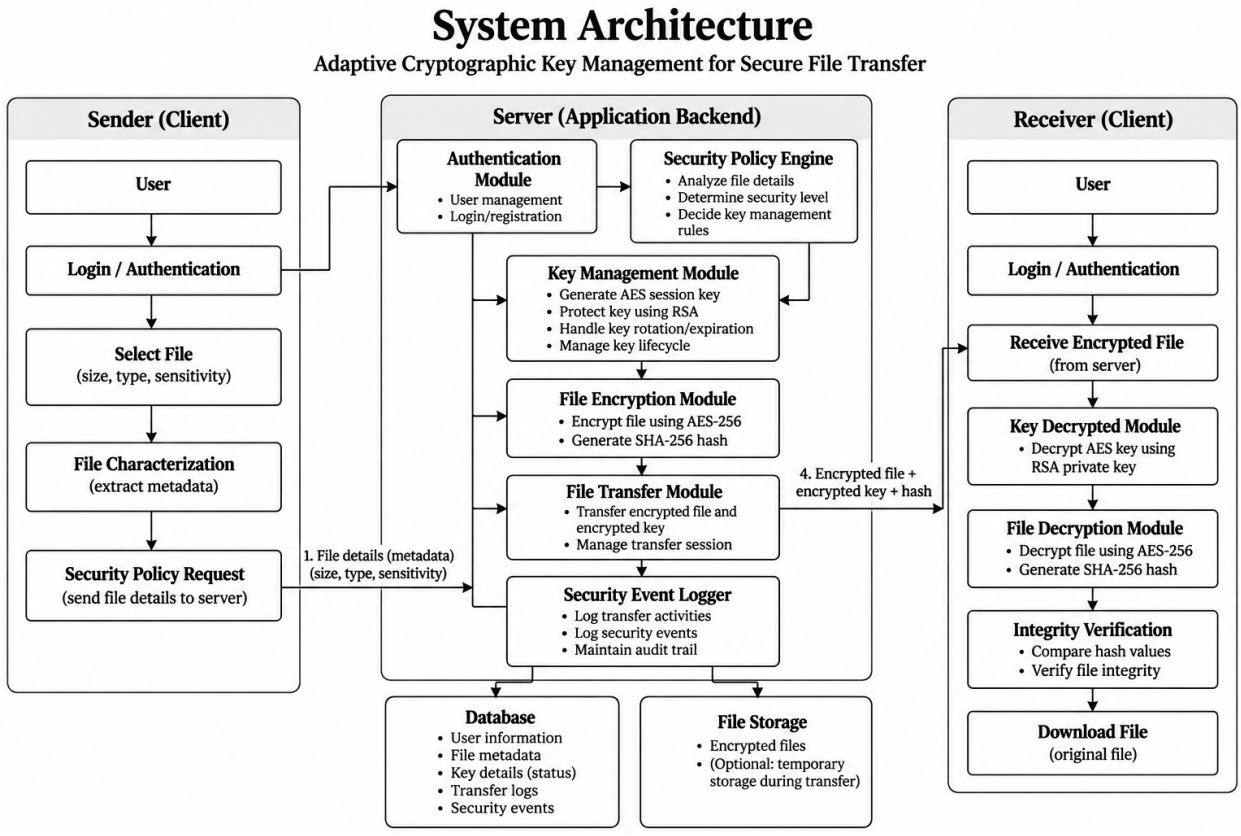
Adaptive Cryptographic Key Management and Integrity Verification for Secure File Transfer

Abstract

Secure file transfer is important for protecting confidential information from unauthorized access, data modification, and key compromise during transmission. Existing systems commonly use encryption algorithms such as AES for data encryption and RSA for secure key exchange. However, simply combining multiple cryptographic algorithms does not provide adaptive key management based on the characteristics and security conditions of each file transfer. Therefore, this project proposes an Adaptive Cryptographic Key Management and Integrity Verification Framework for Secure File Transfer. The proposed system analyzes the file and transfer context, including file size, file sensitivity, session duration, key validity, and security events, and uses this information to determine appropriate key-management actions. For each transfer, a unique AES session key is generated to encrypt the file, while RSA is used to securely protect the session key. SHA-256 is used to generate and verify the file integrity value so that unauthorized modification or corruption can be detected during transfer. The system also manages the complete lifecycle of session keys by supporting key generation, expiration, rotation, and invalidation. Security events such as invalid keys, failed authentication, and integrity failures are recorded for monitoring and auditing. A web-based application will be developed to provide secure file upload, encryption, key management, file transfer, decryption, integrity verification, and security monitoring through a dashboard. The proposed adaptive approach will be compared with a conventional fixed key-management approach using different file sizes and transfer conditions. Performance will be evaluated using encryption time, decryption time, key-generation time, transfer throughput, memory usage, key-management overhead, and integrity-detection capability. The project aims to provide a practical and adaptive approach to secure file transfer while demonstrating the security and performance benefits of effective cryptographic key management.

System Architecture

The system consists of Sender (Client), Server (Application Backend), and Receiver (Client). The server performs authentication, adaptive security-policy evaluation, key management, encryption, transfer management, logging, storage, decryption support, and integrity verification.



Main Components and Functions

Main Component	Function
Authentication Module	Manages user registration, login, and authentication before file transfer.
File Characterization	Extracts file size, type, sensitivity, and other relevant metadata.
Security Policy Engine	Analyzes file and transfer context and determines appropriate key-management rules.
Key Management Module	Generates AES session keys, protects them using RSA, and manages key rotation, expiration, and lifecycle.
File Encryption Module	Encrypts files using AES-256 and generates the SHA-256 integrity value.
File Transfer Module	Transfers the encrypted file, protected session key, and integrity information securely.
Security Event Logger	Records transfer activities, security events, and maintains an audit trail.
Receiver / Decryption Module	Uses the RSA private key to recover the AES key and decrypts the received file.
Integrity Verification	Compares integrity values to detect file modification or corruption.
Database	Stores user information, file metadata, key status, transfer logs, and security events.
File Storage	Stores encrypted files and, when required, temporary transfer data.

Core Cryptographic Operations

• **AES-256:** Encrypts the actual file using a unique session key.

• **RSA:** Protects the AES session key for secure key transfer.

• **SHA-256:** Generates an integrity value used to detect file modification or corruption.

• **Adaptive Key Management:** Controls session-key generation, expiration, rotation, and invalidation according to file and transfer context.

Main Workflow

1. User authentication and file selection.

2. File characterization and security-policy evaluation.

3. AES session-key generation and RSA key protection.

4. AES-256 file encryption and SHA-256 integrity generation.

5. Secure transfer of encrypted file, protected key, and integrity information.

6. RSA key recovery, AES decryption, and integrity verification at the receiver.

7. Security-event logging and key lifecycle management.

Evaluation

The adaptive approach will be compared with a conventional fixed key-management approach using different file sizes and transfer conditions. Evaluation metrics include encryption time, decryption time, key-generation time, transfer throughput, memory usage, key-management overhead, key-rotation overhead, and integrity-detection capability.